

Zombie Memory Cgroup 이슈와 논의 중인 해결 방안

유형곤 (Harry Yoo)

Linux Kernel Developer (Memory Management) @ Oracle

Who am I

유형곤 (Harry Yoo)

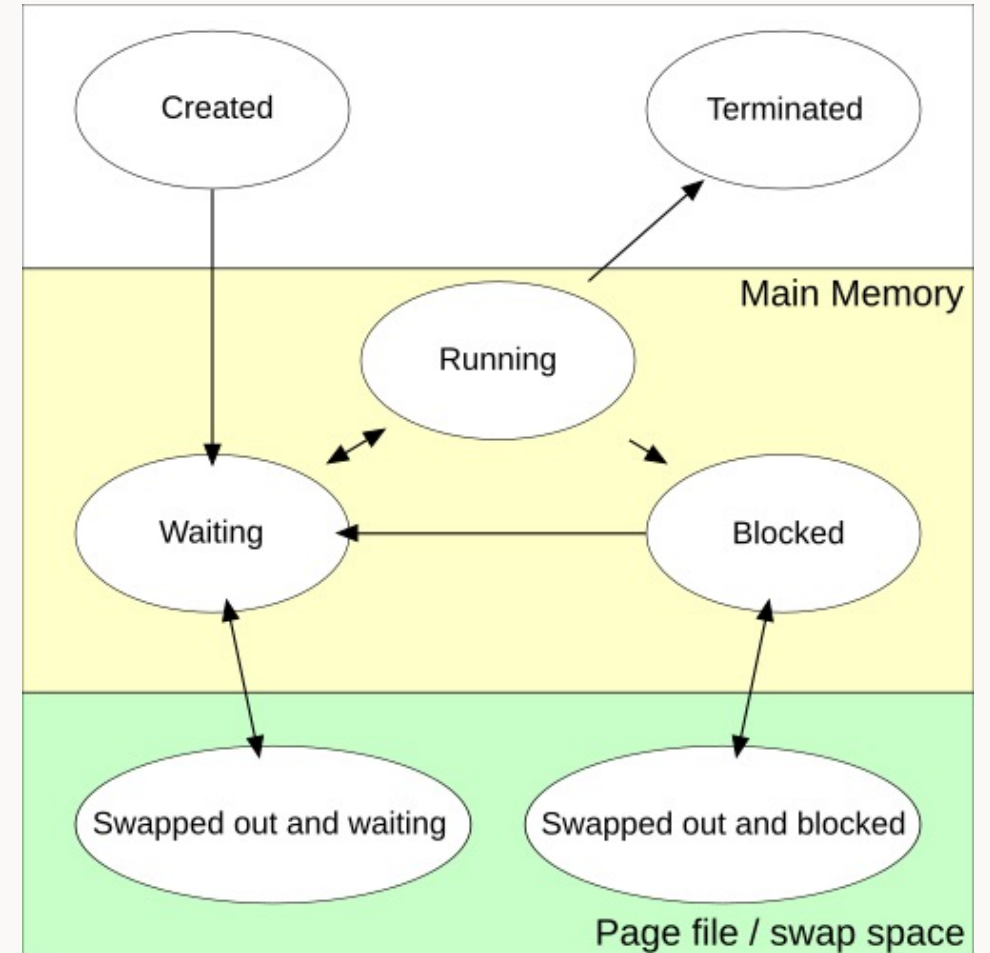
- Linux Kernel Developer (Memory Management) @ Oracle
- 주로 업스트림 및 다운스트림 (Oracle UEK) 커널의 메모리 관리 기능 개발
- Slab Allocator & Reverse Mapping 서브시스템 공식 리뷰어
- 그 외 관심 분야: Memory Cgroup, MGLRU, HugeTLB, HWPoison, ..., etc.

Zombie Memory Cgroup이란

Memory Cgroup은 무엇이고 언제 Zombie가 되는가

Recap: Zombie Process

프로세스가 `exit()`을 호출하여 종료되면
좀비 프로세스 (Terminated state)가 되며,
부모가 `wait()`을 하기 전까지 남는다.
(e.g., 부모가 자식이 왜 죽었는지 알기 위해)



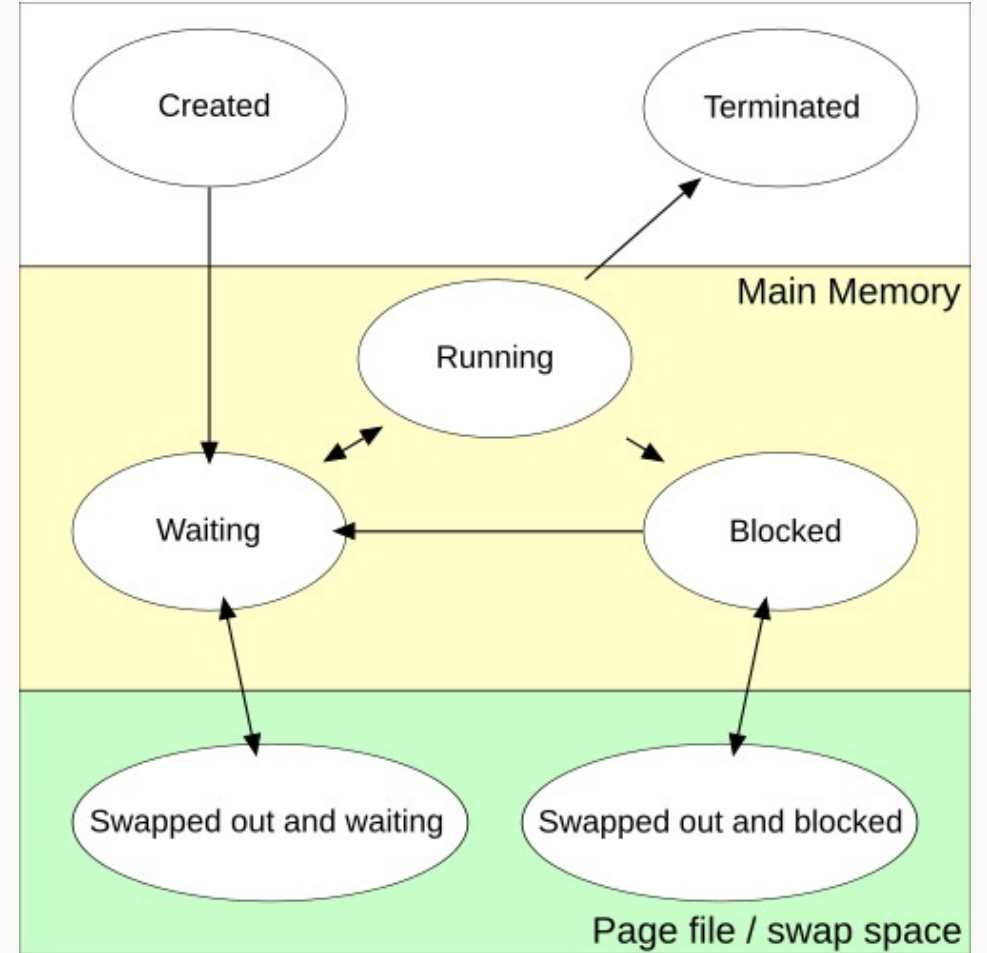
프로세스 상태 다이어그램 [1]

[1] https://en.wikipedia.org/wiki/Process_state

Recap: Zombie Process

프로세스가 `exit()`을 호출하여 종료되면
좀비 프로세스 (Terminated state)가 되며,
부모가 `wait()`을 하기 전까지 남는다.
(e.g., 부모가 자식이 왜 죽었는지 알기 위해)

부모가 `wait()`을 호출하지 않으면 이 정보가
커널 영역에 계속 남고, 이게 계속 쌓이면서
메모리를 낭비하게 된다.



프로세스 상태 다이어그램 [1]

[1] https://en.wikipedia.org/wiki/Process_state

Memory Cgroup Overview

- cgroup은 프로세스들을 그룹으로 관리하여 자원의 사용량을 추적 및 제한하는 기능

Memory Cgroup Overview

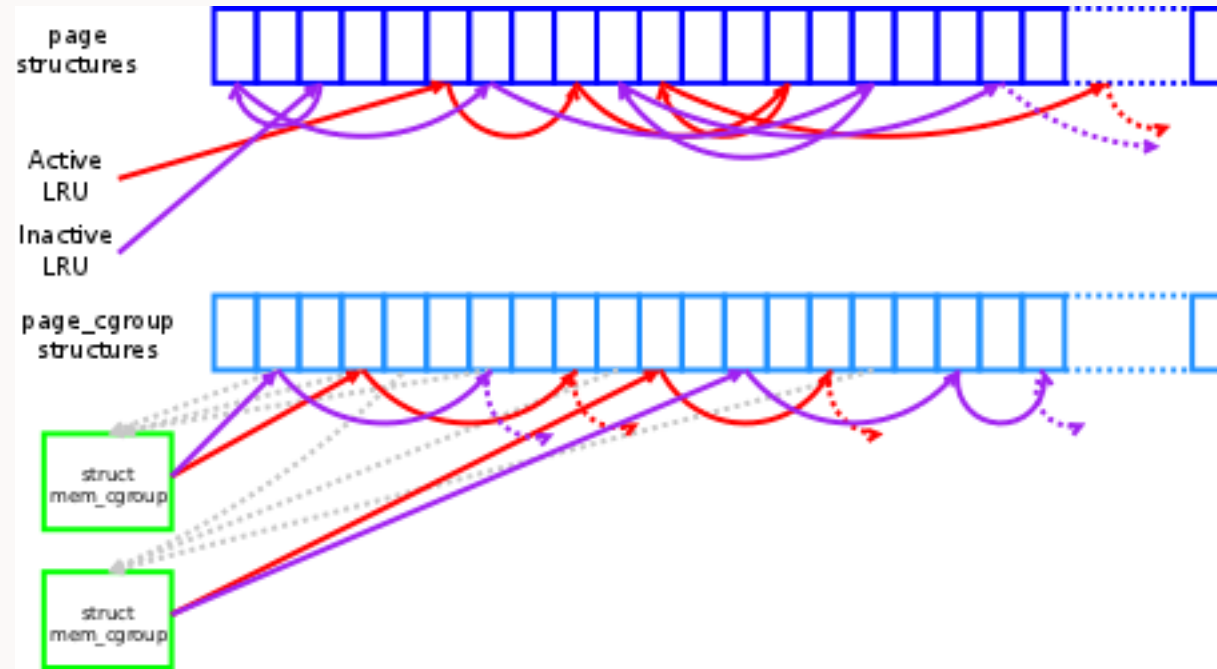
- cgroup은 프로세스들을 그룹으로 관리하여 자원의 사용량을 추적 및 제한하는 기능
- CPU, 메모리 사용량, IO등 각 자원별로 컨트롤러가 존재하며,
컨트롤러는 자원의 사용량을 추적(accounting)하고 제한(limit)할 수 있다.

Memory Cgroup 개요

- cgroup은 프로세스들을 그룹으로 관리하여 자원의 사용량을 추적 및 제한하는 기능
- CPU, 메모리 사용량, IO등 각 자원별로 컨트롤러가 존재하며,
컨트롤러는 자원의 사용량을 추적(accounting)하고 제한(limit)할 수 있다.
- 메모리 컨트롤러를 사용하는 cgroup을 "memory cgroup"이라고 부른다.

Per-Memory Cgroup LRU List

Memory Controller를 사용하는 경우, 각 cgroup의 메모리 사용량을 추적하며 어떤 페이지들을 할당했는지를 Per-Memcg LRU 리스트로 관리한다.



Per-Memcg LRU list [2]

[2] <https://lwn.net/Articles/443241/>

Tricky part: Charging Shared Pages

LRU 리스트가 Per-Memcg라면,

여러 Memory Cgroup 사이에 공유되는 페이지는 어느 LRU에 속해야 할까?

Tricky part: Charging Shared Pages

LRU 리스트가 Per-Memcg라면,

여러 Memory Cgroup 사이에 공유되는 페이지는 어느 LRU에 속해야 할까?

→ 나이브한 방식: 페이지를 가장 먼저 할당한 Memory Cgroup이 페이지를 소유한다.

→ 여기서부터 Zombie Memory Cgroup 문제가 시작된다. ★★★★★

The Zombie Memory Cgroup Problem

- Cgroup에 속한 모든 프로세스가 종료되면 사용자는 Cgroup을 제거할 수 있다.
- 하지만 Memory Cgroup은 LRU에 속한 모든 페이지가 회수되기 전까지는 메모리에서 해제될 수 없다. (각 페이지는 Memory Cgroup의 refcount를 증가)
- 따라서 오랫동안 Memory Cgroup이 메모리에 남게 된다.

해결 방법

그래서 어떻게 이 문제를 해결할 수 있을까?

Solution 1: **Recharging** shared pages to other memcgs

Recharging: memcg가 offline될때, 해당 memcg가 소유한 모든 페이지를, 해당 페이지를 공유하는 다른 memcg에게 소유권을 이전한다.

- 문제점:
 - 소유권을 이전받는 memcg 입장에서는 메모리를 할당하지도 않았는데 메모리 사용량 증가 (charging이 언제 될지 예측 불가능)
 - 어떤 memory cgroup에게 소유를 이전해야 하는지 명확하지 않음

패치 시리즈: [RFC PATCH 0/8] memory recharging for offline memcgs

Solution 2: **Reparenting** pages to the parent memcg

Reparenting: memcg가 offline될때 해당 memcg가 소유한 모든 페이지를 계층구조상 부모 memcg에게 소유권을 이전한다.

- 문제점:
 - Reparenting을 계속 하다보면 결국 root memcg에게 소유권이 이전된다. (root memcg는 accounting을 하지 않기 때문에 결국에는 accounting을 탈출하게 된다.

패치 시리즈: [PATCH RFC 00/28] Eliminate Dying Memory Cgroup

Status Quo

현재 업스트림 논의 상태

Consensus: Reparenting

현재 업스트림에서는 Reparenting을 채택하기로 합의가 되었는데, 이유는 다음과 같다.

1. 현재의 Semantic도 Memory Cgroup이 사라지면 다른 Memory Cgroup으로 자원이 이동하는 게 아니라 부모 그룹에 여전히 속하기 때문에 현존하는 Semantic을 크게 바꾸지 않는다.
2. Recharging과 Reparenting 모두 장단이 있지만, Recharging의 단점이 너무 크다 (언제 Charging이 될지 예측 불가능).
3. 커널 메모리의 (Slab) 경우에는 Reparenting이 되도록 구현 되어있기 때문에 일관성을 맞춰야 한다.

Consensus: Reparenting

Bytedance의 Muchun Song, Qi Zheng을 중심으로
LRU 상의 메모리에 대해서도 Reparenting을 구현하는 시리즈가 개발 중

현재 논의중인 부분은 **MGLRU**를 단순하고 효과적으로 지원하는 것.
Traditional LRU와 다르게 MGLRU는 부모와 자식의 Generation 수가
다를 수 있어서 Reparenting할때 이를 단순하면서도 효율적으로 해결해야 함.



Any Questions?

Thank you!

Additional Slides

How Bad Is The Zombie Memory Cgroup Problem?

- **Factor 1.** memory cgroup 하나의 메타데이터가 얼마나 큰가?
 - 리눅스 커널 5.4 기준
 - struct mem_cgroup -> **2.6 KiB**
 - struct mem_cgroup에서 사용하는 percpu data -> **1.5 KiB per CPU**
 - struct cgroup -> **0.9 KiB**
- => CPU 개수에 따라 memcg 하나당 수 ~ 수백 KiB의 메모리 필요

How Bad Is The Zombie Memory Cgroup Problem?

- **Factor 2.** memory cgroup이 얼마나 자주 생성/삭제 되는가?
 - systemd를 사용하는 시스템에서는 cgroup이 아주 빈번하게 생성 및 삭제됨
 - Users, Applications (Chrome, Slack, ...), SSH sessions, ...
 - 심지어는 systemd timer를 사용해도 새로운 cgroup이 생성됨